



Laboratório de Engenharia de Sistemas e Robótica

Av. dos Escoteiros, 2 - Costa do Sol, João Pessoa - PB

Documento informativo para o Fin Ray gripper versão 0.1

Preparado por: Ronaldo Urquiza Herculano Filho
Data: Setembro de 2026

OBJETIVO:

Descrever como está implementada a primeira versão do Fin Ray gripper proposta ao Laboratório de Engenharia de Sistemas e Robótica.

AO UTILIZADOR:

Este documento apresenta a versão inicial do sistema, desenvolvido como fruto de um projeto de Iniciação Científica.

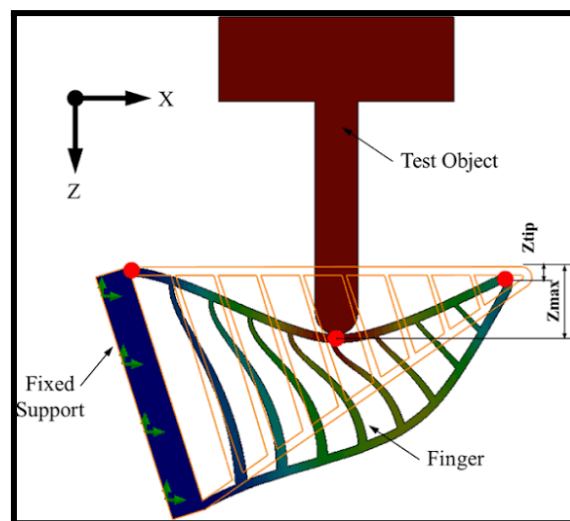
O que se detalha a seguir é o primeiro protótipo funcional gerado após a finalização do projeto.

Ressaltamos que, por ser uma versão preliminar, instabilidades (bugs) podem ocorrer. Logo, otimizações contínuas e o ajuste fino (*fine-tuning*) dos parâmetros estão em processo de desenvolvimento.

1. Por que o modelo Fin Ray foi escolhido?

Alta adaptabilidade e eficiência

O efeito Fin Ray se destaca por sua capacidade de se conformar passivamente ao formato do objeto durante a apreensão: ao encostar em algo, a estrutura flexiona e "abraça" o contorno da peça em vez de aplicar força pontual, distribuindo o contato ao longo de toda a superfície de agarre. Dessa forma, isso permite manipular objetos de geometrias irregulares ou frágeis sem a necessidade de controle de força complexo ou sensoriamento tátil sofisticado.



2. Como isso foi implementado para o Gazebo Classic?

Modelo de corpo pseudo-rígido

É a técnica de aproximar uma estrutura contínua e flexível como uma cadeia de segmentos rígidos conectados por juntas permitindo a simulação em softwares de corpos rígidos como o Gazebo Classic.

Cada segmento (ou, informalmente, dedo) da garra foi dividido em 9 partes chamadas de costeletas.



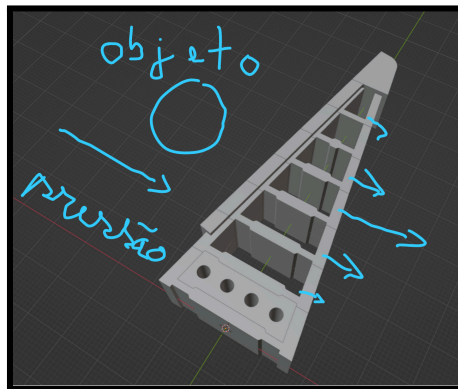
3. Como isso funciona no Gazebo Classic?

Definição de juntas

A primeira costeleta é marcada como de revolução para controlar abertura e fechamento de cada segmento da garra.

O segurador de parafuso é marcado como fixo envolto pela primeira costeleta.

Demais costeletas são prismáticas que se deslocam com limite fixo em um eixo mediante pressão ao contato de um objeto.

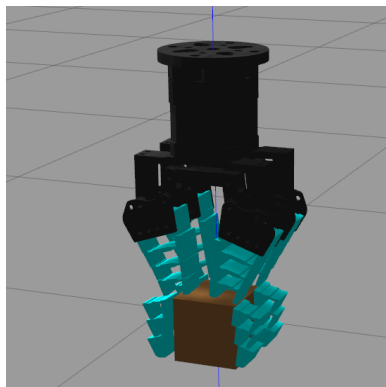


4. Como o agarre é garantido no Gazebo Classic?

IFRA LINK ATTACHER

É evidente que mesmo com a garra se deformando com sucesso no objeto o agarre não pode ser confiado apenas na colisão do gazebo, logo, um plugin chamado [IFRA LINK ATTACHER](#) foi utilizado, portanto, **você precisará dele na sua pasta src junto com o pacote Fin Ray.**

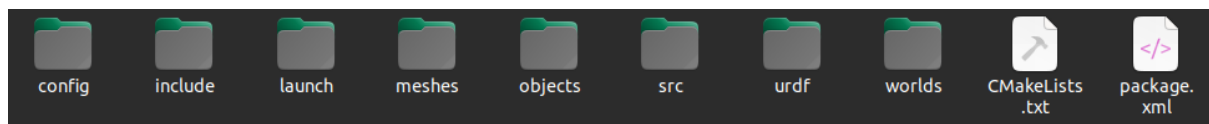
Esse plugin cria uma junta fixa temporária com o objeto que se deseja agarrar que só acontece após o comando de fechamento da garra. E, obviamente, ao abrir, a junta é desfeita e o objeto cai.



5. Como está estruturado as pastas do projeto

Organização

Pasta/Arquivo	Utilidade
config	Arquivos de configuração dos controllers (controllers.yaml), define quais juntas são comandadas e como o ros2_control se conecta ao Gazebo
include	-
launch	Arquivos de launch (.launch.py) que sobem o Gazebo, o robot state publisher, os spawners e os controllers de uma vez
meshes	Malhas 3D (.stl) de cada peça da garra (costeletas, pontas, seguradores), usadas na visualização do URDF
objects	Modelo (.sdf) dos objetos de teste usado nos ensaios de preensão dentro do Gazebo
src	Código-fonte C++ do nó gripper_mover.cpp que controla a abertura/fechamento da garra
urdf	Descrição do robô em Xacro (gripper.urdf.xacro), define links, juntas, inércias e propriedades físicas da garra
worlds	Arquivo de mundo do Gazebo (fin_ray.world), define o cenário/ambiente onde a simulação roda, com uma mesa de café para ser excluída e testar gravidade (se a garra segura ou não o objeto no ar)
CMakeLists	Script de build do CMake/colcon: declara dependências, compila os nodes e instala os arquivos do pacote
package.xml	Manifesto do pacote ROS2: nome, versão, dependências (linkattacher_msgs etc.) e metadados exigidos pelo ament



6. Como executar

Tutorial de execução

Você irá precisar de 3 janelas de terminal.

Primeira janela - Launch

```
> cd [ws que pacote se encontra]
> colcon build
> source install/setup.bash
> ros2 launch fin_ray_description gazebo.launch.py
```

Segunda janela - Nó de movimentação

```
> cd [ws que pacote se encontra]
> source install/setup.bash
> ros2 run fin_ray_description gripper_mover
```

Terceira janela - Envio de informações

```
> ros2 topic pub --once /comando_garra std_msgs/msg/String "data: 'fechar'"
```

ou

```
> ros2 topic pub --once /comando_garra std_msgs/msg/String "data: 'abrir'"
```

Janela opcional - Visualização no RViz

```
> ros2 launch urdf_tutorial display.launch.py
model:=$HOME/finray_ws/src/fin_ray_description/urdf/gripper.urdf.xacro
rvizconfig:=$HOME/finray_ws/src/fin_ray_description/config/view_gripper.rviz
```

* muda mediante nome do ws

7. Como está estruturado o URDF

URDF

A descrição da garra é feita em Xacro (gripper.urdf.xacro), não em URDF puro: em vez de escrever manualmente os ~36 links e ~40 juntas que compõem os 4 dedos (9 costeletas cada), o arquivo define duas macros reutilizáveis: `contato_soft` e `costeleta_prismatica` e as instancia repetidamente, passando só o que muda de uma costeleta pra outra (massa, inércia, posição do centro de massa e nome da malha).

Materiais (visualização no RViz)

soft_blue	Todas as costeletas e pontas dos 4 dedos
preto_claro	Base da garra e os 4 seguradores de parafuso

Junta fixa mundo → base

parent	world	Link raiz, fixo no espaço
child	base_garra	Onde a garra é ancorada
origin xyz	0 0 0.96	Altura (m) da base em relação ao mundo
origin rpy	3.14159265 0 0	Rotação 180° em X — deixa a garra de cabeça pra baixo

Macro contato_soft

ref	Link ao qual as propriedades de contato serão aplicadas
material	Cor/material exibido no Gazebo
mu1 / mu2	Coeficientes de atrito de Coulomb nas duas direções tangenciais do contato
kp / kd	Rigidez (kp) e amortecimento (kd) do contato controlam o quanto a superfície afunda antes de reagir
maxVel	Velocidade máxima de correção de penetração entre corpos
minDepth	Profundidade mínima de penetração antes do motor de física aplicar correção

Macro costeleta_prismatica

prefix / num	Identificam a qual dedo e a qual posição na cadeia a costeleta pertence
nome_mesh / parent	Malha STL usada e link pai na cadeia cinemática (cada costeleta é filha da anterior)
m, cx/cy/cz, ixx...izz	Massa, centro de massa e tensor de inércia daquela costeleta específica (vêm do Fusion)
lim_lower / lim_upper	O quanto a costeleta pode deslizar antes de travar no limite
box_x/y/z	Caixa de colisão simplificada (mais leve que a malha STL completa para o solver de física)
spring_stiffness / spring_reference	Mola implícita do Gazebo que empurra a costeleta de volta à posição de repouso

Junta revolvente da base

axis	Eixo de rotação da abertura/fechamento (z)
origin	Posição/orientação de montagem daquele dedo específico na base (os 4 dedos têm origem espelhada em X/Y)
limit lower/upper	Curso angular, dedos opostos têm limites invertidos porque abrem em sentidos contrários

Bloco ros2_control

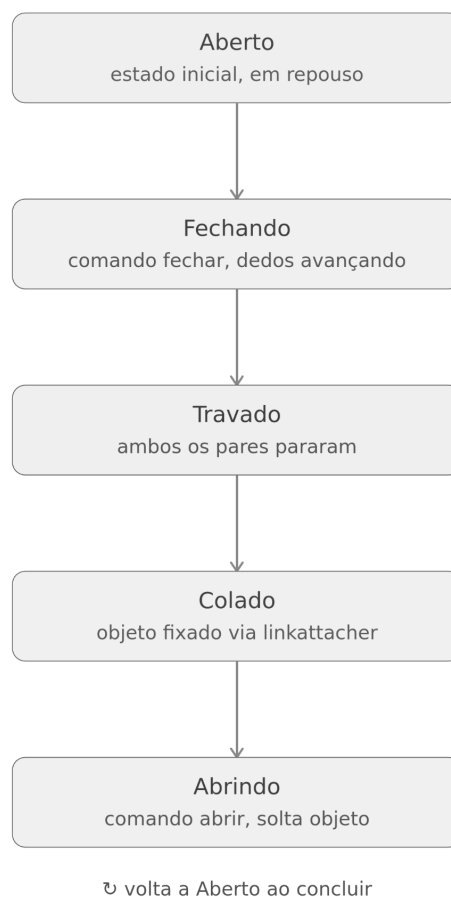
command_interface (position)	Só as 4 juntas revolvente recebem comando, são as únicas ativamente atuadas
state_interface (position/velocity) nas prismáticas	Juntas prismáticas só reportam estado, nunca recebem comando, seu movimento é passivo (mola), não controlado

8. Como está estruturado o CPP

CPP

O `gripper_mover.cpp` é o nó ROS2 responsável por traduzir os comandos de texto `"abrir"` e `"fechar"` (recebidos no tópico `/comando_garra`) em movimento suave das 4 juntas revolvente, detecta quando a garra encontrou resistência (indicando contato com o objeto) e, quando isso acontece, aciona o plugin IFRA Link Attacher para fixar o objeto na garra de forma confiável.

Funcionalmente, o nó se comporta como uma máquina de estados simples.



Tudo está encapsulado numa única classe, `GripperMover`, que herda de `rclcpp::Node`

1. Construtor: configura toda a comunicação ROS2 e inicializa os parâmetros de movimento.
2. Métodos privados: a lógica de negócio (callbacks, controle de movimento, detecção de trava, chamadas ao Link Attacher).
3. Variáveis-membro: o estado do nó, que persiste entre chamadas de callback.

O construtor cria toda a comunicação do nó: um publisher em `/fin_ray_controller/commands` (pra onde vão os comandos de posição das 4 juntas), duas subscriptions (`/comando_garra` para os comandos de texto, `/joint_states` para o estado real das juntas) e dois clientes de serviço (`/ATTACHLINK` e `/DETACHLINK`).

Em seguida define os nomes usados pelo Link Attacher (`modelo_garra_ = "fin_ray_gripper"`, `modelo_objeto_ = "objeto_teste"`, `link_garra_ = "base_garra"`) — esses precisam bater exatamente com os nomes usados no `.world` e no `spawn_entity` do launch file, senão o plugin não encontra os links para colar.

Depois vêm os ângulos de referência: como os 4 dedos se movem em pares espelhados (1&4 giram num sentido, 2&3 no oposto), há dois pares de ângulos aberto/fechado.. O estado inicial começa na posição aberta.

Por fim, os parâmetros que controlam a suavidade do movimento e a detecção de trava:

- `velocidade_` (0.8 rad/s aproximadamente);
- `timer_period_` (20ms — o loop de controle roda a 50Hz);
- `passo_` (quanto avançar a cada tick, calculado a partir da velocidade e do período);
- `limiar_velocidade_travamento_` (abaixo disso a junta é candidata a travada);
- `distancia_minima_antes_de_checar_` (evita falso-positivo logo no início do movimento, quando a velocidade ainda está subindo do zero).

`comando_callback` é uma função roda toda vez que chega uma mensagem em `/comando_garra`.

Se for `"abrir"`, manda os dois pares de volta ao ângulo aberto, reseta as travas e, se havia um objeto colado, chama `chamar_detach()` antes.

Se for `"fechar"`, manda os pares para o ângulo fechado, marca de onde o fechamento começou (`inicio_fechamento_*`, usado depois pra medir se a junta já andou o suficiente antes de checar trava) e zera os contadores de detecção.

Qualquer outro texto é ignorado, apenas logado como aviso.

O ``joint_states`` traz nome, posição e velocidade de todas as juntas do robô (inclusive as prismáticas passivas). Essa função monta um mapa nome -> velocidade e extrai, para cada par de dedos, a maior velocidade entre os dois.

Logo, o par só é considerado "parado" quando os dois dedos individualmente estiverem devagar, não só a média dos dois.

`aproxima` é uma função pequena e direta que em vez de saltar direto para o ângulo alvo, avança `passo_` por tick na direção certa, e só encosta no alvo quando a diferença restante for menor que um passo.

`checa_travamento` verifica duas condições para um par de dedos cada tick:

- (1) o dedo já percorreu uma distância mínima desde que começou a fechar (`distancia_minima_antes_de_checar_`),
- (2) a velocidade real da junta caiu abaixo do limiar (`limiar_velocidade_travamento_`). Se as duas forem verdadeiras, incrementa um contador; caso contrário, zera o contador. Isso é importante porque evita que um único tick ruidoso (um pico isolado de baixa velocidade que não significa contato real) dispare uma trava falsa.

Só quando o contador atinge `LEITURAS_CONSECUTIVAS_` (5 ticks seguidos, ou seja, 100ms de velocidade consistentemente baixa) a trava é confirmada: o alvo é travado na posição atual, impedindo que o dedo continue tentando fechar contra o objeto.

`timer_callback` é executado a cada 20ms pelo `timer_`, esse método executa os seguintes passos:

1. Chama ``checa_travamento()`` para cada par, independentemente;
2. Aplica um timeout de segurança (`MAX_TICKS_FECHAMENTO_ = 150 ticks = 3 segundos`) se o fechamento demorar demais sem detectar trava;
3. Só chama `chamar_attach()` quando os dois pares já travaram, o objeto ainda não está colado e não há uma tentativa de colagem em andamento;
4. Avança a posição publicada de cada par um passo em direção ao alvo (usa `aproxima()`).
5. Publica o comando final no formato esperado pelo controller: `{atual_1_4_, atual_2_3_, atual_2_3_, atual_1_4_}` respeitando a ordem da lista de juntas do `controllers.yaml` (`dedo1`, `dedo2`, `dedo3`, `dedo4`), com `dedo2/dedo3` recebendo o mesmo valor (``atual_2_3_``) e `dedo1/dedo4` recebendo o outro (``atual_1_4_``), já que cada par se move espelhado.

`chamar_attach` e `chamar_detach` são do `Link Attacher`:

- Ambas seguem o mesmo padrão: primeiro checam se o serviço (``/ATTACHLINK`` ou ``/DETACHLINK``) está disponível, se não estiver, avisam no log que o plugin provavelmente não foi carregado no ``world``.

- Depois montam o request com os nomes de modelo/link definidos no construtor e enviam de forma assíncrona (`async_send_request`) (nó não trava esperando resposta; um lambda roda quando ela chegar)

Em `chamar_attach()`, a flag `tentando_colar_` evita que o `timer_callback()` dispare uma segunda chamada enquanto a primeira ainda não respondeu.

Em `chamar_detach()` isso não é necessário, porque descolar um objeto que já está descolado não causa problema.

Por fim, a main inicializa o contexto do ROS2, cria o nó e entra no loop de eventos (processando timer, subscriptions e respostas de serviço até receber Ctrl+C), e encerra tudo de forma limpa.

Por enquanto o manual se encerra aqui.